

Advice and Recommendations Document

This document analyzes the JabberPoint codebase issues and offers possible solutions and recommendations for upgrading the code using various design patterns.

Code Quality and Conventions

Key Issues:

- No Package Structure:**
All classes reside in the default package, hindering organization and namespace management.
 - Mixed Language Comments:**
The code contains both Dutch and English comments; it should be standardized to English.
 - Legacy Java Patterns:**
**The code uses `Vector<T>` instead of `ArrayList<T>`, old-style for-loops, and mutable static state.
 - Inconsistent Access Modifiers:**
Protected static constants exist in some classes, while others are public.
 - Empty Catch Blocks:**
Several locations silently swallow exceptions without logging or handling.
 - Code Conventions:**
The code lacks consistent conventions, with a mix of camelCase and snake_case. Additionally, bracket placements are inconsistent, and spaces between functions and fields are lacking.
 - Instructions:**
The project does not contain a README markdown file, which is essential for helping developers understand the purpose and functionality of the code.
-

Recommendations:

- Introduce Consistent Naming Conventions:**
Implement a single naming convention for variables and fields for better readability, such as using snake_case for fields and constants.
- Refactor Large Functions:**
For better readability and future debugging, split massive functions into smaller ones instead of creating everything in one constructor. Use the command pattern to separate logic and UI creation.
- Use Custom Error Handling:**
Implement proper exception handling to improve error management. This means introducing proper exception usage to the code-base to describe the error more thoroughly.
- Update Organization and Readability:**
Write instructions and comments in one language and introduce a README markdown

file for future developers.

5. **Introduce consistent access modifiers:**

Make sure to specify the access modifiers to avoid unexpected behavior and bugs.

6. **Implement a Package Hierarchy:** Move all classes from the default package into a logical namespace structure (e.g., `com.jabberpoint.ui`, `com.jabberpoint.model`) to organize the codebase and prevent naming collisions.

7. **Update Organization and Readability:** Translate all Dutch comments to English to standardize the codebase. Introduce a comprehensive README.md file at the root of the repository detailing the project purpose and architecture.

8. **Modernize Java Constructs:** Replace synchronized legacy collections like `Vector` with modern equivalents like `ArrayList`. Convert traditional for-loops to enhanced for-each loops where applicable, and eliminate mutable static state to prevent unpredictable side effects.

9. **Introduce Consistent Access Modifiers:** Enforce the principle of least privilege. Make fields private by default, use protected only when strict inheritance requires it, and expose functionality via public methods rather than direct field access.

10. **Implement Proper Error Handling:** Stop silently swallowing exceptions. Introduce custom exception classes to describe specific domain errors thoroughly, handle them at the correct application layer, and ensure every caught exception is at least logged.

11. **Enforce Java Standard Naming Conventions:** Implement a strict, single naming convention. Standardize variables, fields, and methods to camelCase (strictly avoiding snake_case for standard fields). Classes should be PascalCase, and UPPER_SNAKE_CASE should be reserved exclusively for static final constants. Standardize bracket placement and whitespace.

Suggested Design Patterns to Use:

1. Menu and Keyboard Controller Commands

- **Current Status:**

`MenuController` contains inline anonymous `ActionListener` classes with duplicated patterns. `KeyController` uses a large switch statement. Both violate SRP (handles construction, execution, I/O, and error handling) and OCP (adding commands requires modification).

- **Recommended Pattern:** Command Pattern

- **Solution:**

Encapsulate each action as a standalone object with an `execute()` method. This enables reuse across menu/keyboard, unit testing without GUI, and undo/redo support.

2. SlideItem Creation (TextItem, BitmapItem)

- **Current Status:**

`XMLAccessor.loadSlideItem()` uses if-else chains to instantiate `SlideItem` types; `saveFile()` uses `instanceof` checks. This violates OCP (new types require multiple modifications) and SRP (`XMLAccessor` knows too much about concrete implementations).

- **Recommended Pattern:** Factory Method Pattern

- **Solution:**
Centralize creation logic in a factory class. New item types can be added by extending the factory without modifying `XMLAccessor`.
- 3. **DemoPresentation and XMLAccessor. loading**
 - **Current Status:**
 - In DemoPresentation class, slide creation is handled manually. In XMLAccessor, the loadFile function contains for loops and contains a big amount of code. Additionally, the saveFile function uses an unstructured approach to build the presentation for saving. This makes the code less readable and makes it more complex for future presentation implementation and features.
 - **Recommended Pattern:** Builder pattern
 - **Solution:**
 - Introduce a Builder class which will reduce duplicate code, improve readability make it easier for more complex presentation creation in the future.

Summary

Implementing these recommendations and design patterns will enhance code quality, improve maintainability, and facilitate future development.

Component	SOLID Violation	Pattern	Category	Key Benefit
Controllers	SRP, OCP	Command	Behavioural	Decoupling; undo/redo
SlideItem Creation	OCP, SRP	Factory Method Pattern	Creational	Extensible item types
DemoPresentation and XMLAccessor loadFile & saveFile	OCP	Builder Pattern	Creational	Configurable; efficient